Moving Metrics, Not Data: A Low-Cost Ingestion Architecture for Proactive Business Intelligence Across Heterogeneous Sources

Palla Chetana Reddy, Varshini Yadav Nasaram, and Obulagari Nandini

Department of Computer Science and Engineering, School of Technology, Woxsen University

Capstone Project-I (B.Tech Computer Science and Engineering, 7th Semester)

Dr. K RajKumar

October 7, 2026

Abstract

Proactive business-intelligence systems have to read data from whatever systems a company already runs, yet managed connector platforms are priced by data volume, self-hosted alternatives are heavy to operate, and copying large exports over FTP is slow. This paper describes and evaluates the ingestion layer of KAIROS, an open-source system that monitors business metrics and explains anomalies causally. The design follows one principle: move the computation, not the data. Five protocol-level adapters (SQL, SFTP/FTP, files and object storage, REST, and push) replace per-product connectors. Aggregation runs inside the source database or in a small edge agent placed beside large files, every adapter reads incrementally and writes idempotently, and a schema-inference module proposes the mapping for a new source. When a raw file has to move, a transfer service sends compressed chunks over parallel streams, checks each chunk with SHA-256 and resumes after interruptions. In a live demonstration with four simulated company systems, a 100,305-row order table was summarized at the source and transferred as 19.5 KB, and the FTP adapter fetched only a newly added 130-byte file. On a 105.6 MB export of two million rows, with each connection capped at 50 Mbit/s, the transfer service was 6.4 times faster than single-stream FTP, an interrupted transfer re-sent only the missing half, and edge aggregation reduced the payload to 192.7 KB. The results suggest that the volume of data moved, more than the protocol, determines ingestion cost in metric monitoring.

Keywords: data ingestion, business intelligence, edge aggregation, file transfer, ELT, schema matching

Moving Metrics, Not Data: A Low-Cost Ingestion Architecture for Proactive Business Intelligence Across Heterogeneous Sources

A system that watches business metrics on its own is only as good as its access to data. In a typical small or mid-sized company, orders sit in an ERP database, marketing figures arrive as spreadsheet exports, support statistics live in a SaaS tool behind a REST API, and a logistics partner drops a file on an FTP server each night. KAIROS, the system this paper belongs to, collects such metrics every 15 minutes, detects anomalies, traces them to a causal origin and sends a verified alert. Its analysis depends on a steady supply of fresh, correctly mapped data from all of these places.

When the project was reviewed, the evaluation panel raised the obvious objection. Data can come from any source, every company uses a different set of systems, connectors are heavy and expensive, and FTP takes a long time to move large files. One reviewer also noted that tools exist which transfer even large files directly. These are fair points. The standard industrial answer is an extract-load-transform (ELT) platform: either a managed service that maintains hundreds of connectors and bills by monthly active rows (Fivetran, n.d.), or an open-source platform that has to be deployed and maintained by the user (Airbyte, n.d.). Both copy source tables more or less as they are into a warehouse. For a monitoring system that needs a few dozen KPI series, this is a large and costly detour.

The observation behind this paper is that a metric monitor does not need raw rows at all. Fifty KPIs sampled every 15 minutes amount to under 5,000 rows a day, while the order table they are computed from may hold millions. The idea of shipping the computation to the data rather than the data to the computation is old; it underlies MapReduce (Dean & Ghemawat, 2008) and, more recently, edge computing (Shi et al., 2016). This paper applies it to business-

intelligence ingestion and asks three questions. How can one system read from arbitrary company systems without writing a connector for each product (RQ1)? How can it handle large data and large files without slow bulk transfers (RQ2)? And what does this cost (RQ3)? The paper reviews related work, describes the ingestion design, reports a live demonstration and a transfer benchmark, and discusses limitations.

Literature Review

Data Integration Pipelines

Data integration has moved from hand-written extract-transform-load jobs to ELT platforms that replicate source tables into a central warehouse and transform them there. Kleppmann (2017) describes the properties that make such pipelines dependable, notably idempotent writes and replayable logs, which allow a failed step to be retried without corrupting data. Managed platforms implement these properties and maintain connectors on behalf of customers, and their pricing reflects the volume replicated (Fivetran, n.d.). Open-source platforms remove the licence cost but run each sync in containers and need an orchestration layer (Airbyte, n.d.). Change data capture tools read a database's transaction log and stream row changes as they happen (Debezium, n.d.). They are efficient for keeping a replica current, but they still move row-level data, which a monitor of aggregate metrics does not need.

Bulk File Transfer

The File Transfer Protocol dates from the early days of the Internet and moves each file over a single TCP connection (Postel & Reynolds, 1985). Its secure successors run over SSH (Ylonen & Lonvick, 2006). The throughput of a single TCP stream is bounded by the congestion window, the round-trip time and the loss rate; Mathis et al. (1997) showed that steady-state throughput falls in proportion to the round-trip time and to the square root of the loss probability.

On long or lossy paths one stream therefore leaves most of the link idle. Hacker et al. (2002) measured the effect of opening several TCP sockets in parallel and found substantial gains on lossy wide-area networks. Commercial products such as IBM Aspera avoid TCP's limits altogether with a rate-controlled protocol over UDP (IBM, n.d.). Their licence cost puts them out of reach for most small firms. Two further techniques shrink what has to be sent. General-purpose compressors such as Zstandard trade little CPU time for large reductions on text formats (Collet & Kucherawy, 2021), and columnar formats store each column separately so that readers fetch only what they need (Melnik et al., 2010).

Processing Near the Data

MapReduce scheduled computation on the machines that already held the input blocks, so that large datasets did not cross the network (Dean & Ghemawat, 2008). Edge computing extends the same reasoning to devices and branch sites, processing data where it is produced and sending summaries upstream (Shi et al., 2016). Embedded analytical databases make this practical on ordinary hardware. DuckDB runs inside the host process, reads CSV and Parquet files directly and aggregates data larger than memory (Raasveldt & Mühleisen, 2019), so a single script can summarize a multi-gigabyte export on a laptop.

Schema Mapping

Every new source raises the question of which columns mean what. Rahm and Bernstein (2001) surveyed automatic schema matching and classified approaches by whether they use schema information, instance data or both. In business intelligence the problem is usually solved by hand in a semantic layer, which is precise but slow to set up. For metric monitoring the mapping is narrower than general schema matching: the system needs a timestamp, the metric

columns and the shape of the table. That narrower task can be inferred from a sample of rows with simple, checkable rules.

Research Gap

Existing tools treat ingestion as replication: they copy rows, priced or provisioned by volume, and leave aggregation to the warehouse. Transfer accelerators address large files but are proprietary. No reviewed approach combines protocol-level adapters, aggregation at the source, incremental and idempotent collection, automated mapping and a free, resumable transfer path for the cases where raw data must move. The design below fills this gap for metric monitoring.

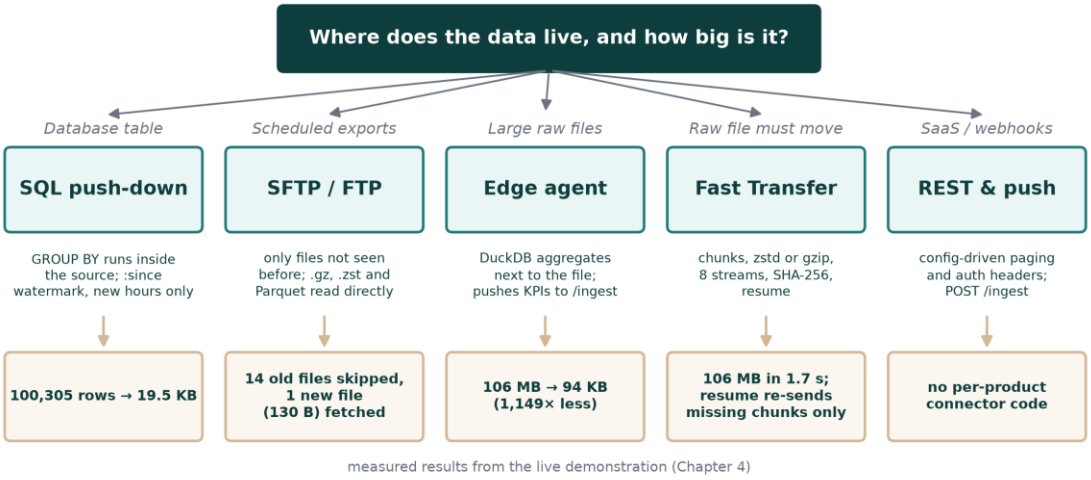
Method

System Context

KAIROS runs five agents in sequence under a LangGraph state machine (LangChain, n.d.). The first agent, the collector, is the subject of this paper. It turns every source into rows of the form (metric, value, timestamp) and stores them in DuckDB under the key (source, metric, timestamp). The remaining agents detect anomalies, learn a causal graph with DYNOTEARS (Pamfil et al., 2020), write a report that is checked against the data, and send the alert. Figure 1 summarizes how a source is routed according to where its data lives and how large it is.

Figure 1

Ingestion Routes by Data Location and Size



Note. The bottom row shows results measured in the live demonstration reported in the Results section.

Protocol Adapters

Instead of one connector per product, the collector implements adapters for the protocols through which almost all business systems expose data (Table 1). Each adapter is a small class, between 100 and 200 lines, with one method that receives the source's saved state and returns new rows together with an updated state. A new company is connected by configuration rather than code: a source entry names the adapter, the connection details, and optionally the query or column mapping. Secrets are read from environment variables named in the configuration, never from the file itself.

Table 1

Protocol Adapters and What Each One Moves

Adapter	Systems covered	Incremental mechanism	What travels
SQL	PostgreSQL, MySQL, SQL Server, Snowflake, SQLite and ERP or CRM systems built on them	Query parameter :since bound to the last timestamp	Result of an aggregating query
SFTP / FTP	Any system that can export a scheduled file	Name, size and modification time of files already processed	New files only, often compressed
Files and object storage	CSV, Excel, Parquet and JSON on disk, S3 or HTTPS	Timestamp filter pushed into DuckDB	Only needed columns and new rows
REST	SaaS and in-house APIs	Configurable since parameter and paging	JSON records after the watermark
Push	Webhooks, in-house applications, the edge agent	Sender decides what is new	KPI records posted to the API

Note. A Google Sheets adapter is also available for small teams that keep metrics in spreadsheets.

Incremental and Idempotent Collection

Each adapter keeps a small state object between runs: a timestamp watermark for databases, files and APIs, and a record of processed files for SFTP and FTP. A file counts as processed by name, size and modification time, so a file that is overwritten with new content is read again. Files younger than a configurable age, or without a matching completion marker, are skipped so that half-written uploads are never read. For sources that aggregate in their own query, the last time bucket is re-read on the next run, because it may have been incomplete; the upsert simply replaces the earlier value. Because every write is an upsert on (source, metric, timestamp), repeating a sync never duplicates data, which is the property that makes retries safe (Kleppmann, 2017). Sources are polled in parallel, each with its own timeout, and a failing source is reported without blocking the others.

Aggregation at the Source

For databases, the configured query performs the aggregation, for example summing order amounts and counting orders per hour for all hours after the watermark. The database does the scanning and only the hourly totals cross the network. For files, the adapter hands the path or glob to DuckDB, which reads compressed CSV, Parquet or JSON directly, filters by timestamp and groups rows into time buckets without loading the file into memory (Raasveldt & Mühleisen, 2019). The same code path works for files on S3 or behind HTTPS.

Edge Agent

Some companies cannot expose a database and hold their data as large exports on internal servers. For them, KAIROS provides an edge agent: a single Python script that runs inside the company network next to the data. It aggregates a raw export with DuckDB in one pass, posts the resulting KPI records to the KAIROS ingest endpoint, and remembers the last bucket it sent, so the next run reads only new rows. The agent needs outbound HTTPS only, which is usually easier to approve than inbound database access.

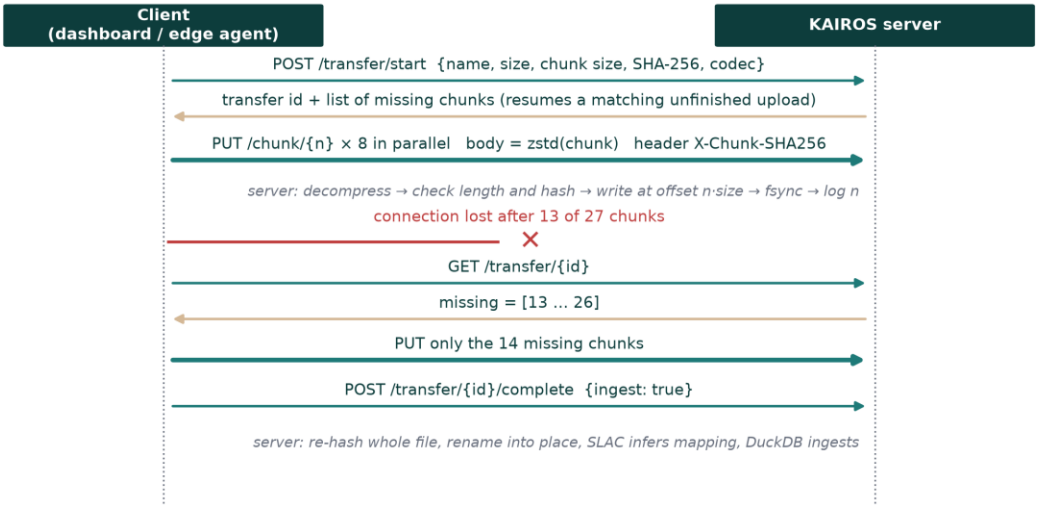
Fast Transfer

When the raw file itself is needed, for example for a one-time historical load, the transfer service replaces FTP (Figure 2). The client splits the file into chunks of 1 to 32 MB, compresses each chunk with gzip or Zstandard, computes its SHA-256 hash and uploads several chunks at the same time over separate HTTP connections. The server decompresses each chunk, checks its length and hash, writes it at its offset in a pre-allocated file, flushes it to disk and only then records it as received. If the connection drops, the client asks the server which chunks are missing and sends only those. When all chunks have arrived, the server hashes the whole file again before accepting it, and the file can be ingested at once using the mapping proposed for it.

The same protocol is used by the command-line client and by the dashboard, where the browser compresses and hashes chunks with built-in web APIs.

Figure 2

Fast Transfer Protocol With Resumption After a Dropped Connection



Schema Inference

A module called Semantic Layer Auto-Construction (SLAC) proposes the mapping for a new source from a sample of rows. It picks the timestamp column by type, name and parse rate; decides whether the table is long (one row per metric and time) or wide (one column per metric); excludes identifier columns such as order numbers, which are numeric but meaningless to sum; and, when timestamps are irregular or repeated, concludes that the table holds raw records and proposes hourly aggregation with a record count. The proposal is shown to the user, who can accept or edit it. An optional language-model step adds business meaning, such as whether a rise in the metric is good or bad, but the mapping itself never depends on it.

Evaluation Design

Two evaluations were run on a Windows laptop with Python 3.13. The first was a live demonstration with four simulated company systems: an SQLite ERP database holding 100,305 raw order rows, a folder of three weekly marketing CSV exports, an FTP server receiving one compressed logistics file per day (14 files), and a paged REST API serving support metrics behind a bearer token. All four were registered through configuration and synchronized through the KAIROS API. The second was a transfer benchmark on an export of two million order rows (105.6 MB as CSV). The KAIROS server and a standard FTP server ran as separate processes on the same machine. To emulate a wide-area link, every connection was paced to at most 50 Mbit/s, and the same pacing was applied to FTP and to Fast Transfer. Wall-clock time and bytes on the wire were recorded for six methods. In addition, the adapters are covered by automated tests that start a real FTP server and a real HTTP server rather than mocks.

Results

Live Demonstration

Table 2 lists what happened in the demonstration. All four systems were collected in one call, with the ERP source sending 19.5 KB for more than a hundred thousand raw rows because the hourly aggregation ran inside the database. A second sync immediately afterwards moved nothing new from the FTP, file and REST sources. When a new file was dropped on the FTP server, only that file was downloaded. Figure 3 shows the dashboard page that reports these figures for each source.

Table 2

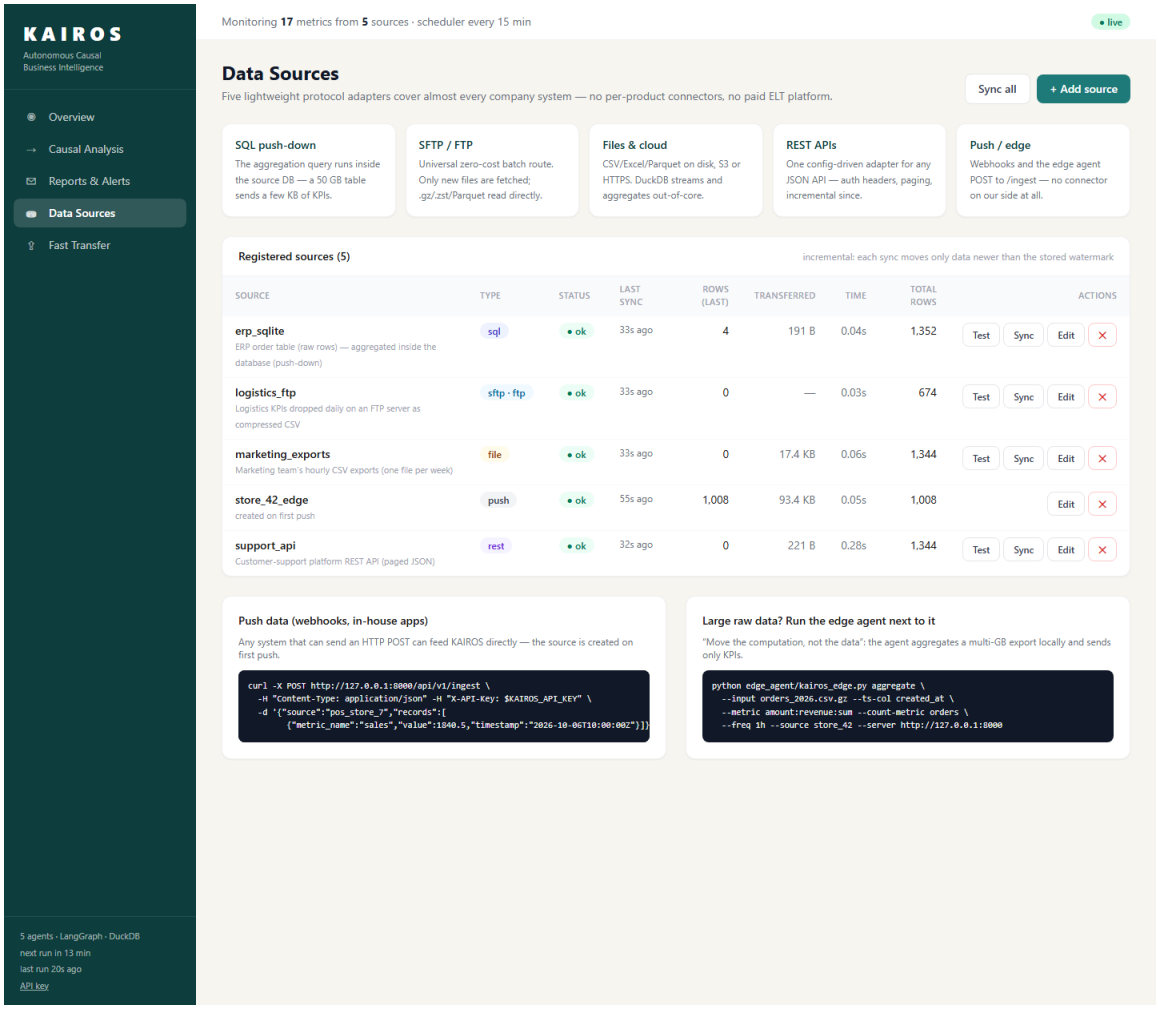
Live Ingestion Demonstration With Four Simulated Company Systems

Step	Result
First sync of all four sources	4,704 KPI rows collected; the ERP source sent 19.5 KB for 100,305 raw rows
Immediate second sync	No new rows from the FTP, file or REST sources; nothing duplicated
New file dropped on the FTP server	Only that file (130 bytes) downloaded; 14 earlier files skipped
Edge agent on a 105.6 MB export	Aggregated in 0.6 s; 94.1 KB pushed, 1,149 times less data
Fast Transfer of the same export	28.7 MB sent in 1.7 s; SHA-256 verified on the server
Connection cut after 13 of 27 chunks	Resumption re-sent only the 14 missing chunks

Note. The ERP database re-sent its four most recent hourly buckets on the second sync, by design, because they may have been incomplete.

Figure 3

Data Sources Page After the Live Demonstration



Note. Each row shows the adapter type, status, rows and bytes moved in the last sync, its duration and the total rows stored.

Transfer Benchmark

Table 3 and Figure 4 give the benchmark results. Plain FTP took 16.9 s to move the 105.6 MB file. Compressing the file before sending it by FTP cut the bytes to 27.1 MB, but compression on one core took 4.8 s, so the total was still 9.2 s. Fast Transfer compressed chunks in parallel and sent them over eight streams, finishing in 2.6 to 2.8 s, about six times faster than

FTP. Zstandard was slightly faster than gzip at a slightly larger size. When the connection was cut half way, the resumed transfer sent only the missing 13.6 MB. Edge aggregation was the fastest of all: it read the two million rows locally and sent 192.7 KB of hourly KPIs, about 550 times fewer bytes than the raw file.

Table 3

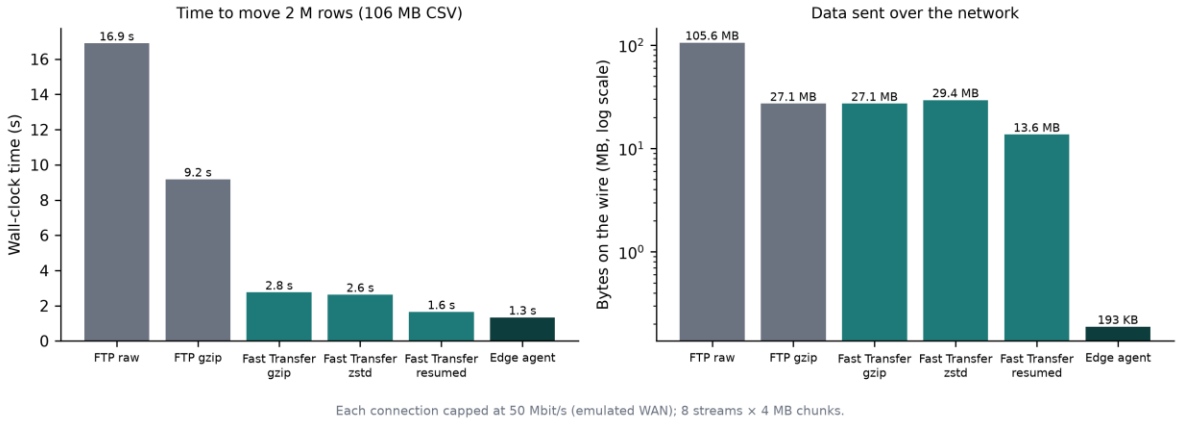
Transfer Benchmark for a 105.6 MB Raw Export

Method	Time (s)	Bytes sent	Speed-up vs. FTP
FTP, single stream, raw CSV	16.90	105.6 MB	1.0
FTP, single stream, gzip (including compression)	9.18	27.1 MB	1.8
Fast Transfer, 8 streams, gzip	2.76	27.1 MB	6.1
Fast Transfer, 8 streams, Zstandard	2.63	29.4 MB	6.4
Fast Transfer, resumed after a 50% drop	1.65	13.6 MB	10.2
Edge agent (aggregate locally, push KPIs)	1.34	192.7 KB	12.6

Note. Each connection was paced to 50 Mbit/s to emulate a wide-area link; both servers ran on the same machine. Byte counts are exact; times vary with hardware and load.

Figure 4

Transfer Time and Bytes on the Wire for Each Method



Note. The right panel uses a logarithmic scale.

Downstream Use

The ingested KPIs fed the rest of the pipeline without any change. In the live run, the detector flagged 71 anomalous points across the four sources, the causal stage named advertising spend and delivery delay as the origins of the two injected incidents, and the full five-agent cycle took 12.4 s. Ingestion therefore did not limit the speed of analysis.

Discussion

Answering the Research Questions

On RQ1, five protocol adapters were enough to connect four quite different systems by configuration alone, which supports the claim that protocols, not products, are the right unit of integration for metric monitoring. On RQ2, the strongest result is that large data rarely needs to travel. Aggregation in the source database or in the edge agent reduced the volume by three orders of magnitude, and for the remaining cases a parallel, compressed and resumable transfer was several times faster than FTP and robust to interruptions. This answers the reviewer's point about tools that move large files directly: the same techniques are available in free software, and they complement rather than replace aggregation at the source.

Cost

On RQ3, the software has no licence cost, and the running cost follows the volume moved. Under row-based pricing, cost grows with the number of rows that change (Fivetran, n.d.). In the demonstration, the ERP source produced 1,344 KPI rows for 100,305 order rows, about 75 times fewer, and the reduction grows with order volume because the number of hourly buckets does not. The remaining costs are a small server for KAIROS and, where used, a few minutes of a staff member's time to approve the edge agent.

Security

The demonstration used plain FTP because it is easy to inspect, but production deployments should use SFTP or FTPS (Ylonen & Lonvick, 2006) with read-only accounts restricted to one folder. Database access should use read-only users, ideally on a replica. The API can require a key, secrets are taken from the environment, and every transferred chunk and file is verified by hash. Aggregation at the source also has a privacy benefit: customer-level records stay inside the company, and only totals leave it.

Limitations

The benchmark ran on one machine with an emulated link, so absolute times on a real wide-area network will differ, although the byte counts will not. The data was synthetic. Aggregated KPIs cannot answer questions that need raw records, such as which customers were affected, so a drill-down still requires access to the source. The edge agent has to be installed inside the company network, which some IT departments will not allow. SLAC uses rules and will need a person to check unusual tables, and change data capture is not yet supported.

Future Work

Planned work includes measurements over real long-distance links, a change-data-capture adapter for databases that cannot run aggregating queries (Debezium, n.d.), columnar Parquet output from the edge agent, and evaluation of SLAC on public datasets with known schemas.

Conclusion

The concerns raised in review were that data can come from any source, that connectors are costly and that FTP is slow for large files. This paper answered them with one principle and a small set of mechanisms. Five protocol adapters, configured rather than coded, connected four different systems. Aggregation at the source and in an edge agent reduced the data moved by

three orders of magnitude. A free, parallel and resumable transfer service handled the cases where raw files must move, about six times faster than FTP. For metric monitoring, the question to ask of a data source is not how to copy it quickly but how little of it needs to be copied at all.

References

Airbyte. (n.d.). *Airbyte documentation*. Retrieved October 6, 2026, from https://docs.airbyte.com/

Collet, Y., & Kucherawy, M. (2021). *Zstandard compression and the application/zstd media type* (RFC 8878). Internet Engineering Task Force. https://doi.org/10.17487/RFC8878

Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified data processing on large clusters. *Communications of the ACM, 51*(1), 107–113. https://doi.org/10.1145/1327452.1327492

Debezium. (n.d.). *Debezium documentation*. Red Hat. Retrieved October 6, 2026, from https://debezium.io/documentation/

Fivetran. (n.d.). *Usage-based pricing: Monthly active rows*. Retrieved October 6, 2026, from https://fivetran.com/docs/usage-based-pricing

Hacker, T. J., Athey, B. D., & Noble, B. (2002). The end-to-end performance effects of parallel TCP sockets on a lossy wide-area network. In *Proceedings of the 16th International Parallel and Distributed Processing Symposium*. IEEE.

IBM. (n.d.). *IBM Aspera*. Retrieved October 6, 2026, from https://www.ibm.com/products/aspera

Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media.

LangChain. (n.d.). *LangGraph documentation*. Retrieved October 6, 2026, from https://langchain-ai.github.io/langgraph/

Mathis, M., Semke, J., Mahdavi, J., & Ott, T. (1997). The macroscopic behavior of the TCP congestion avoidance algorithm. *ACM SIGCOMM Computer Communication Review, 27*(3), 67–82. https://doi.org/10.1145/263932.264023

Melnik, S., Gubarev, A., Long, J. J., Romer, G., Shivakumar, S., Tolton, M., & Vassilakis, T. (2010). Dremel: Interactive analysis of web-scale datasets. *Proceedings of the VLDB Endowment, 3*(1–2), 330–339. https://doi.org/10.14778/1920841.1920886

Pamfil, R., Sriwattanaworachai, N., Desai, S., Pilgerstorfer, P., Georgatzis, K., Beaumont, P., & Aragam, B. (2020). DYNOTEARS: Structure learning from time-series data. In *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics* (pp. 1595–1605). PMLR.

Postel, J., & Reynolds, J. (1985). *File Transfer Protocol (FTP)* (RFC 959). Internet Engineering Task Force. https://doi.org/10.17487/RFC0959

Raasveldt, M., & Mühleisen, H. (2019). DuckDB: An embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data* (pp. 1981–1984). ACM. https://doi.org/10.1145/3299869.3320212

Rahm, E., & Bernstein, P. A. (2001). A survey of approaches to automatic schema matching. *The VLDB Journal, 10*(4), 334–350. https://doi.org/10.1007/s007780100057

Shi, W., Cao, J., Zhang, Q., Li, Y., & Xu, L. (2016). Edge computing: Vision and challenges. *IEEE Internet of Things Journal, 3*(5), 637–646. https://doi.org/10.1109/JIOT.2016.2579198

Ylonen, T., & Lonvick, C. (2006). *The Secure Shell (SSH) protocol architecture* (RFC 4251). Internet Engineering Task Force. https://doi.org/10.17487/RFC4251